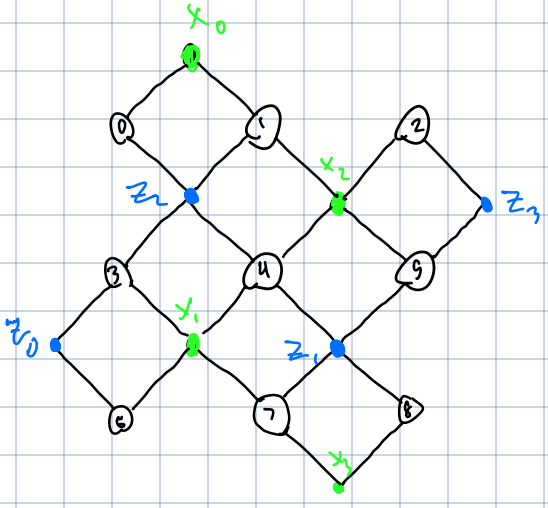
Theory behind QEC Decoders

1. Define the problem

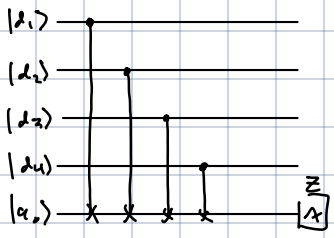
Consider 3x3 surface code: superconducting error correction code topology



Legend:

- (di) : data qubit i
- X_j : X stabilizer j
- Z_k : Z stabilizer k

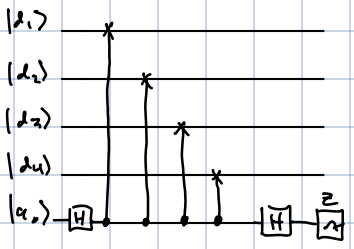
Z stabilizer circuit:



$|d_i\rangle$ = data qubit i

$|a_j\rangle$ = ancilla qubit j

X stabilizer circuit:



* Every $|d_i\rangle$ is connected to ≥ 2 stabilizer circuits.

Each stabilizer circuit takes the adjacent data qubits and runs this algorithm with ancilla. 1 ancilla readout = 1 stabilizer / "detector" readout

→ First parameter: N = num detectors = 8

Consider the faults this circuit can represent directly:

- Pauli (X, Y, Z) error
- Measurement (CNOT) / "hook" error
- Depolarization during idle - in between stabilizer rounds

"represent": the error can be mapped to its original cause by ideal decoder.

Faults it can't represent:

- Coherent phase errors
- Leakage into higher states $|2\rangle$ etc.

→ these errors disrupt representable patterns as noise

2. DEM

DEM = statistical model for syndrome histories

STIM path:

Detector error model

num detectors: $r(d^2-1)$ r: num rounds d: distance

1. Define some p_e of error happening

2. Propagate fault $\vec{e}$ through ckt with given p_e to get (h_e, l_e) pairs ($\vec{e}$ hidden)
   - h_e: detector-flip vector $h_e \in H$: map of detectors flips. $H\vec{e} = \vec{s}$ syndrome
   - l_e: logical-flip vector $l \in O$: map of observable flips. $O\vec{e} = l \hookleftarrow$ per-shot label of observable flip
   - $\vec{e}$: explicitly a vector $e_j \in \{0,1\}^E$, $E :=$ num mechanisms (possible errors) on given topology. In STIM case: $\vec{e}$ sampled to give data (X, y) for training where X = stack of syndromes, y = source of truth $O\vec{e} = l \hookleftarrow y$

3. Outline ex. flow *mod 2 ops

   Prep $|0\rangle$
   Sample(p_e) $\rightarrow \vec{e}$ true
   Propagate errors: $s = H\vec{e}\,|_{\text{mod 2}}$ stabilizer detector events
   $y = O\vec{e}\,|_{\text{mod 2}}$ label

IN PRACTICE:
p_e unknown. Estimate from syndrome data $\rightarrow$ once p_e characterized, obtain DEM

3. Mechanisms

Consider some 3x3 topology.

Pauli flip mechanisms: $\{X, Y, Z\}$ per data qubit. "Clifford depolarization"
CNOT error in stabilizer circuit: consider false-flip or phase. both map to X or Z errors. Show:

> CNOT$(|d_0\rangle \otimes |a_{x_0}\rangle)$ fails to flip $|a_{x_0}\rangle$ when it should have. Trivial to show this is equivalent to CNOT $(X|d_0\rangle \otimes |a_{x_0}\rangle)$. ∴ we represent it as X error on $|d_0\rangle$
>
> Arbitrary phase drift $e^{i\phi Z}$, $\phi := \frac{\theta}{2}$: $I\cos(\phi) + iZ\sin(\phi)$. For some round of syndrome measurement $e^{i\phi Z}|\psi\rangle$ collapses to Z flip w/ probability $\sin^2(\phi)$

∴ DEM can map phase or CNOT errors to some independently firing Pauli error

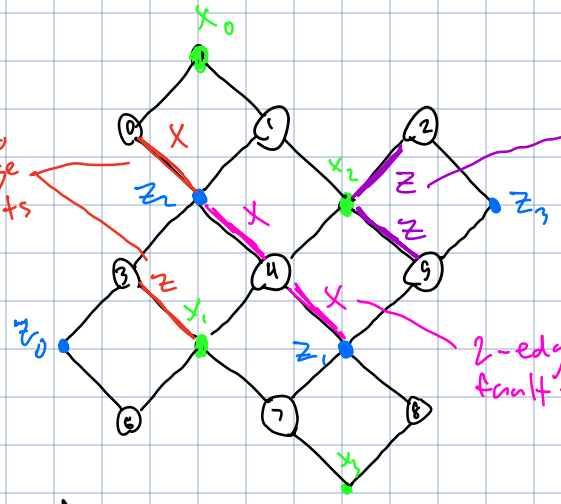
Consider $h_e \in \{0,1\}^N$ = the vector of stabilizers which some Pauli error propagates to. For given circuit topology we define the indices of h_e to map to given detector. 3x3 case say $0 \rightarrow \{x_0 \rightarrow x_3, z_0 \rightarrow z_3$.

E.g. write h_e for X flip on $|d_0\rangle$, Z on $|d_3\rangle$: (And build some intuition for other faults)

(1)

$$h_e = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

two 1-edge faults



Z fault on d_2 and d_5 both map as a single edge to X_2. Mathematically these are duplicate columns in H_X, underlined below

Z-edge fault: X on d_4

Every error in the domain of this code can be represented as $\{X, Y, Z\}$ on $|d_i\rangle$

→ Write every $h_{e\{X,Z\}}$ for X and Z errors as matrices H_X, H_Z

$$H_Z = \begin{array}{c} \\ X_0 \\ X_1 \\ X_2 \\ X_3 \end{array} \begin{array}{c} \begin{array}{cccccccccc} d_0 & d_1 & d_2 & \cdots \end{array} \\ \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & \underline{0} & \underline{0} & 0 & \underline{0} & \underline{0} & 1 & 1 \end{pmatrix} \end{array} \qquad H_X = \begin{array}{c} \\ Z_0 \\ Z_1 \\ Z_2 \\ Z_3 \end{array} \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ \underline{0} & \underline{0} & 1 & 0 & \underline{0} & 1 & 0 & \underline{0} & \underline{0} \end{pmatrix} \quad (2)$$

$Y = -iXZ$ is represented as X and Z error same time: "triggers" both X and Z stabilizers

Note purple underlined columns: duplicate detector signatures for X/Z error on given $|d_i\rangle$. Collapse duplicates into 1.

→ Write full H for $r = 1$. Code-level (X, Y, Z), spacelike (no time edges) check matrix:

$$H = \begin{pmatrix} H_X & 0 & H_X \\ 0 & H_Z & H_Z \end{pmatrix} \quad X, Z, Y \text{ columns} \quad (3)$$

→ Parameter L: number of mechanisms. At rnd 1, $3 \cdot 9 = 27 - 4$ dup. $= 23$

L increases round by round. Define $P \in \{X, Y, Z\}$ arbitrary Pauli operator

True error $\vec{e}$ is unknown and hidden. $\vec{e} \in \{0,1\}^L = \mathbb{F}_2^L$

$e_i = e_{i,t-1} \oplus e_{it}$. XOR of detector resident in previous round

Consider time + space errors:

Space (1D): Data error between rounds. $|d_i\rangle \mapsto P|d_i\rangle$. Every later round sees error P
∴ only 1 XOR event fires.

Time (2D): Ancilla measurement error. Since ancilla is reset every rnd the error stream looks like 0 1 0 so $\bigoplus_i = 1$ (1 ∴ 2 XOR events)
(1 = error, 0 = reset)

Time + Space (2D): Hook CNOT error mid-round. Some checks have seen correct state while others haven't. New check sees at t, old check sees it at $t+1$. Errors span ≥ 2 detectors and 2 rounds

∴ With each round possible L increases. Brute force calculate and find

r	1	2	3	4	5	... for given 3x3 surface code topology + STIM pipe.
L	23	107	219	331	443	

∴ since h changes, H also must change to match

4. Mechanism → error → syndrome → prediction → correction

Consider simple case at r=1.

We have L=23 mechanisms for d=1 data qubit. For a given experiment we have 1 observable (memory X or Z in STIM).

For any arbitrary physical error, decoder attempts to predict which correction should be applied. We can define e as the true error and $\hat{e}$ as the prediction and then define some residual:

$$\gamma = e + \hat{e} \quad e, \hat{e} \in \mathbb{F}_2^L \quad (4)$$

where $\gamma = \vec{0}$ in a successful prediction and $\gamma \neq \vec{0}$ for incorrect $\hat{e}$. $\gamma \neq 0$ signifies a logical error. The goal then is to predict $\hat{e}$ given $s = He$ by optimizing some cost function describing most likely e from characterized p_e and then map $\hat{e} \to \hat{l}$ where $\hat{l} \in \{0,1\}$ is whether e corresponds to whether error occured in given basis.

$$C_l(s) = \{e : He = s|_{\text{mod } 2}, Oe = l|_{\text{mod } 2}\} \quad (6a) \qquad D : s \mapsto \hat{l} \text{ some decoder}$$

$$\text{If } H\hat{e} = s \text{ then } H\gamma = He + H\hat{e} = \emptyset \to \gamma \in \ker H \quad (6b)$$

Let S be the stabilizer group as operators on the codespace. The codespace is the +1 eigenspace of its stabilizers. The set of X, Z stabilizers in 3x3 surface code "generate" S. Let $\{g_1, \ldots, g_a\}$ be the set of these generators. Then every element of S is a product of generators i.e. every stabilizing set in surface code is a product of operators g_i.

$\leftrightarrow X_i X_i = I \leftrightarrow g_i g_i = I \leftrightarrow (X : z_j)(X : z_j) = I$ but $X_i X_j \neq I$ because they interact with different data qubits.

From $g|\psi\rangle = |\psi\rangle \; \forall g \in S$ it follows that any generator of S acts trivially on $|\psi\rangle$.

Observe that a successful error correction has net trivial affect on $|\psi\rangle$.

A net trivial affect on $|\psi\rangle \leftrightarrow l = \emptyset$ where $O\hat{e} = l$.

From (5) the residual γ of a successful prediction $\in \ker H$

If $l = \emptyset$ for success then

$$O\gamma = \emptyset = O\hat{e} + Oe = \hat{l} + l, \; l \in \{0,1\} \quad (7).$$

For $O\gamma = \emptyset$, $\gamma \in \ker O$

Then it follows that a successful prediction is constrained by:

$$\gamma_{\text{correct}} = \hat{e} + e \in (\ker H \cap \ker O) = S. \quad (8).$$

where $O :=$ the map of observables flipped → logical error produced. $1 \times L$ for

L mechanisms, 1 observable (X or Z).

In practice we don't know C. $\therefore$ We use this relationship as the target for optimization of some decoder D to predict $\hat{l} \in \{0, 1\}$ based on syndrome s.

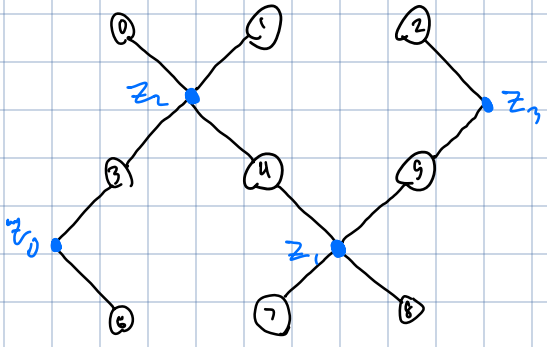
If $D: s \mapsto \hat{l}$ is correct then $\hat{l} + l = \emptyset \leftrightarrow O\hat{c} + Oc = \emptyset \leftrightarrow H\hat{c} + Hc = \emptyset$

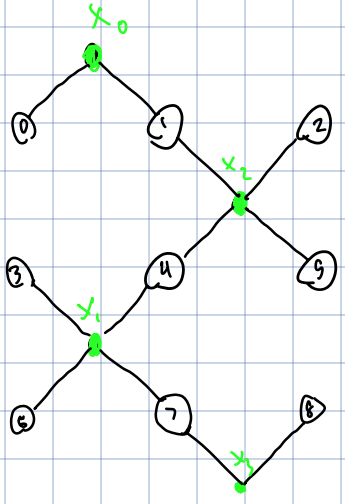
$\leftrightarrow \hat{c} \in S \leftrightarrow$ trivial action on codespace $\leftrightarrow$ error is stabilized/corrected.

5. How to construct logical qubit from surface code

A natural question to ask is which c_i actually translate into error on logical qubit given that there are 9 data qubits on the lattice. Require that all sets of Z checks commute with logical X and all X checks commute with logical Z (must be independent). Then write the support of X and Z stabilizers across the whole circuit.

$$S_X = \{\{0,1,3,4\}, \{2,5\}, \{3,6\}, \{4,5,7,8\}\} \quad | \quad S_Z = \{\{0,1\}, \{1,2,4,5\}, \{3,4,6,7\}, \{7,8\}\}$$





These are generally operators $Z(a)$, $X(b)$ where

$$a, b \subseteq \{1, \ldots, 9\}$$

Define commutation of $Z(a)$ with $X(b)$:

$Z(a)$ and $X(b)$ commute iff $a \cap b$ is 0 or even. (9)

- Can trivially verify that the set of Z and X stabilizers commute.

Motivate (6): $ZZ = XX = I \leftrightarrow$ doing nothing. 2 or 0 commute

$\therefore$ Logical operators x_L and z_L must anticommute because $XZ = -ZX$. So $x_L z_L$ must not be I in order for the circuit to be useful.

I won't derive it here for brevity but find that any operator chain $Z(a)$, $X(b)$ of length ≥ 3 satisfies this set of constraints. $\therefore$ We choose

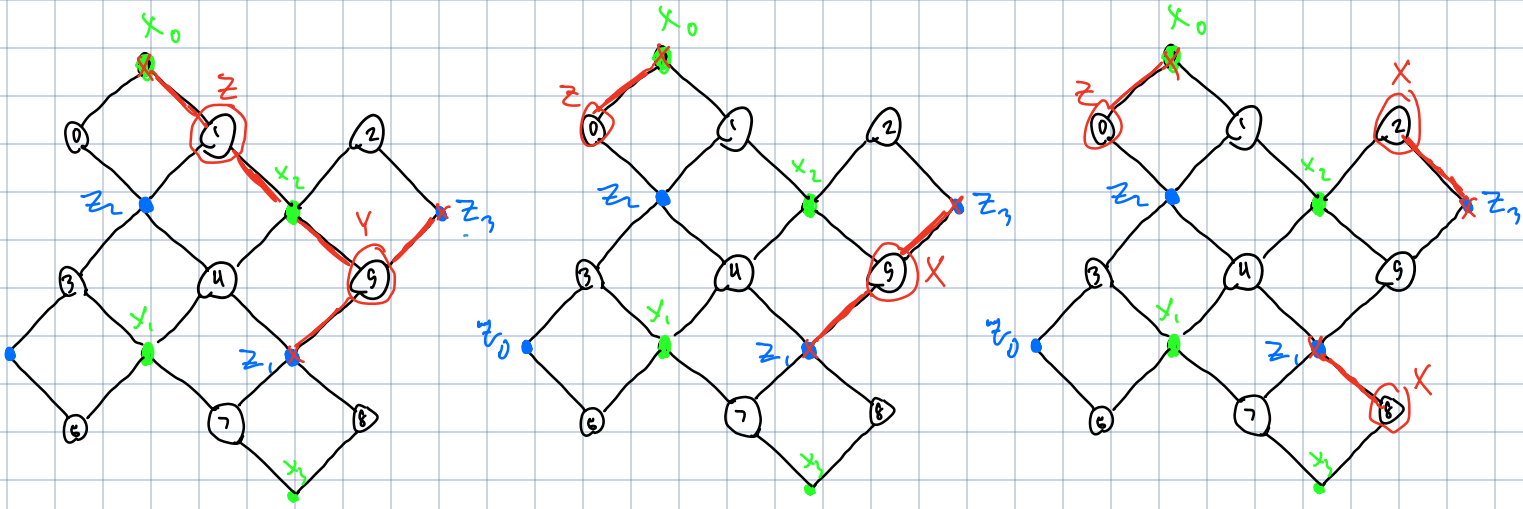
$$z_L = \{d_0, d_1, d_2\} \text{ and } x_L = \{d_0, d_3, d_6\} \quad (10)$$

- Can verify z_L or any horizontal set of 3 commutes with X stabilizers. Same for vertical x_L and Z stabilizers.
- STIM uses this convention to mark whether given syndrome $\leftrightarrow$ error on logical qubit or not.

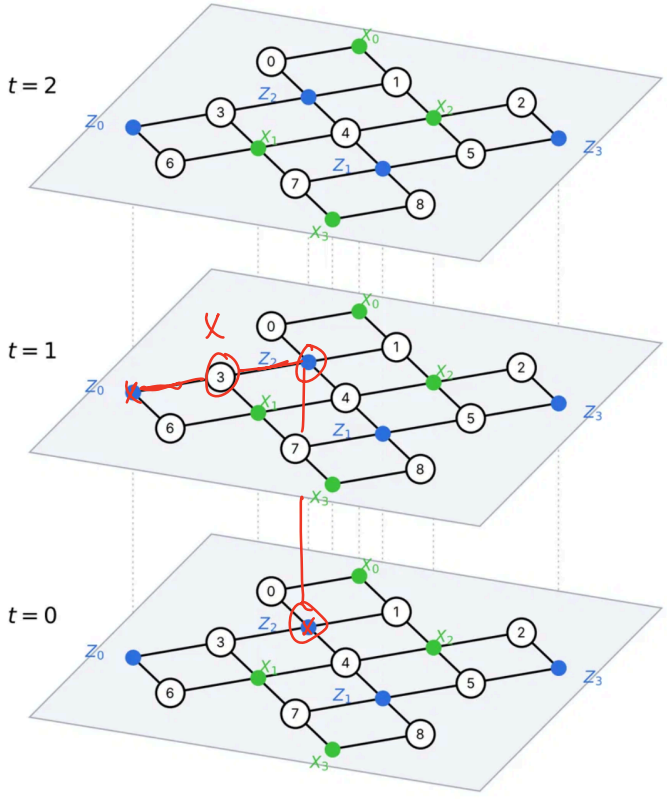
6. Types of mechanisms

Here I will demonstrate examples of different error classes mapping to different edge sets. 'X' on stabilizer ⟷ detector fires (triggered by XOR as in DEM)

E1: Three space-type faults give rise to the same syndrome assuming they share the same initial condition. Left → right more → less complex.



E2: 1 timelike fault → space fault → logical error



$t=2$: With no new errors no detector fires.

$t=1$: After resetting the detector should fire again from 1 → 0 change but an X error on d_3 keeps it at 1 and also fires error on $Z_0 \cdot Z_2$ stays constant

$t=0$: A measurement error on ancilla Z_2 causes a fault → detector picks up the 0 → 1 change on XOR and fires error.

7. Decoding Algorithms

Let's look more closely at $H = \begin{pmatrix} H_x & H_x \\ H_z & H_z \end{pmatrix}$. Write it explicitly:

$$H = \begin{array}{c} \\ z_0 \\ z_1 \\ \vdots \\ \\ x_0 \\ x_1 \\ \vdots \end{array} \begin{array}{c} \begin{array}{cccccccccc} X\text{ on } & d_0 & d_1 & \cdots & & Z\text{ on } & d_0 & d_1 & \cdots & \end{array} \\ \left(\begin{array}{cccccccccc} 0&0&0&1&0&0&1&0&0 & \\ 0&0&0&0&1&1&0&1&1 & \\ 1&1&0&1&1&0&0&0&0 & \\ 0&0&1&0&0&1&0&0&0 & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \end{array}\right. \\ \left(\begin{array}{cccccccccc} & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ 1&1&0&0&0&0&0&0&0 & \\ 0&0&0&1&1&0&1&1&0 & \\ 0&1&1&0&1&1&0&0&0 & \\ 0&0&0&0&0&0&1&1&0 & \end{array}\right. \end{array} \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array} \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array} \begin{array}{c} Y\text{ on } d_0\ d_1 \cdots \\ \left(\begin{array}{cccccccccc} 0&0&0&1&0&0&1&0&0 & \\ 0&0&0&0&1&1&0&1&1 & \\ 1&1&0&1&1&0&0&0&0 & \\ 0&0&1&0&0&1&0&0&0 & \\ 1&1&0&0&0&0&0&0&0 & \\ 0&0&0&1&1&0&1&1&0 & \\ 0&1&1&0&1&1&0&0&0 & \\ 0&0&0&0&0&0&1&1&0 & \end{array}\right. \end{array} \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array} \tag{11}$$

We can think of H as an incidence matrix which maps errors on qubit d_i to event on detector x/z_j. Recalling that X errors trigger Z stab., $Z \to X$ stab., $Y \to X$ and Z, we see that each nonzero "region" of H characterizes a type of Pauli fault. $H_{ij} = 1$ if fault i flips detector j. It can represent code-level space faults but note that there is no time edges. Code-level because it's constructed of edge map H_z, H_x, not modeling other faults

Important distinction: H is not an operator in the sense that it translates the error itself to a syndrome we observe. Recall that prediction of $\hat{l}$ (did a fault happen) $\in \{0,1\}$ is based on observed syndrome s and subject to $Hc = s$ where c is the unknown error. "Subject to" is a posterior over the distribution that renormalizes it to the region of observed s i.e. without $Hc = s$ the algorithm has no way to understand which class of c could have produced the observed s, so it would calculate over the whole space arbitrarily. H sets region boundaries, s tells us what region of error space we are in.

To put some meath to my prose:

$$P(s|c) = \mathbb{1}[Hc = s] \leftrightarrow s \text{ is a deterministic func. of the faults } c$$

$$\to P(c|s) \propto P(c)\,\mathbb{1}[Hc = s], \quad P(c) = \prod_i p_i^{c_i}(1-p_i)^{1-c_i}, \quad p_i \text{ known prior} \tag{12}$$

* Note that this $P(c)$ models all faults c as generated by i.i.d. noise $\therefore$ decoding is only as good as the model. This becomes important later.

For $l \in \{0,1\}$ the label of whether error occurs or not, O observable map: we know from section 4 that we can split error space into trivial set S (8) and nontrivial set

$$E = \{c : c \in \ker H \setminus \ker O,\ Hc = s, Oc = l\} \tag{13}$$

which describes logical errors.

Look again at (12). See product over i: this operation finds probability of some c by considering p_i at each support $c_i = 1$ and $1-p_i$ at $c_i = 0$.

Consider one observable:
Each c has a label ℓ which encodes whether a logical fault happens or not. Class membership is determined by condition $Oc = \ell$. ∴ A decoder must predict this ℓ given that c is always hidden. For a given syndrome the region $Hc = s$ has various c, some with $\ell = 0$ and some with $\ell = 1$. To predict $\hat{\ell}$ you might take the max of a sum of all c within the two classes. Write it:

$$\hat{\ell} = \arg\max_{\ell} \sum_{c} \Pr(c), \quad c \in C_\ell(s), \quad C_\ell(s) \text{ defined as in (6a)} \quad (14)$$

To calculate this explicitly you would need to enumerate all $c \in C_\ell(s)$. $C_\ell(s)$ has condition $Hc = s$, H has dimension $n \times L$. $n :=$ num detectors, $L :=$ num mechanisms.

$$n = r(d^2-1), \quad A := \begin{pmatrix} H \\ O \end{pmatrix}. \quad A \text{ has each mechanism labeled by } O(j) \text{ whether logical error occurs at } j.$$

Rewrite $C_\ell(s)$:

$$C_\ell(s) = \{c : Ac = b\}, \quad b = \begin{pmatrix} s \\ \ell \end{pmatrix} \text{ syndrome with label. } A \in \mathbb{F}_2^{(n+1)\times L}$$

We know already that L increases with rounds and d. I got Claude to fit a polynomial to find closed form in L:

$$L_Z = (23d-13)(d-1)r - 17d^2 + 13d - 3, \quad L_X = L_Z + 2, \quad L = L_Z + L_X \quad (16)$$

Solution space of $C_\ell(s)$:

$c \in c_0 + \ker A$ where c_0 is trivial solution

$\dim \ker A = L - \operatorname{rank}(A)$.

In $\mathbb{F}_2$, k-dim space has 2^k elements. ∴ there are $2^{L-\operatorname{rank}(A)}$ solutions

$$|C_\ell(s)| = 2^{L-\operatorname{rank}(A)} \quad (17).$$

By (16) this exponent grows polynomially with d and linearly with r ($\hat{=} \propto d^2$).
For realistic hardware distance (e.g. Willow $d = 7$) and experiment rounds ($30 \le r \le 100$) solving (14) naively quickly becomes intractable just based on the number of cases to consider.
There exist methods to leverage planar structure of code-level noise to solve MLD for planar Pauli noise (arXiv:1405.4883) but these break down under depolarizing, measurement and circuit-level (e.g. SI1000) noise.
Additionally, this model described by (12) and (13) assumes i.i.d. noise and DEM (section 2) is one way to generate/characterize data under this model. Most decoder algorithms are either slow or inaccurate. ∴ The goal of some neural decoder is to find solutions which balance accuracy with speed. A machine learning algo has fixed

inference computational load $\leftrightarrow$ speed whereas a decoding algo must search solution space in some way which has slow+fast cases $\therefore$ not constant. Neural decoder could also learn patterns beyond the model which establishes prior for a static decoder.

8. Example decoding algorithm: Minimum Weight Perfect Matching (MWPM)

In general, to exhaustively calculate all possible edge permutations / their probabilities is too expensive. We leverage some intuition about surface code to simplify the decoding problem:

1. Syndrome s contains information about chit topology + incidences of errors through $Hc = s$. We can think of a syndrome being the boundary of an error chain c (i.e. which detectors error c triggers).
2. In surface code, each data qubit touches at most 2 checks in each basis X, Z.
3. Error suppression techniques allow us to model noise as i.i.d. Pauli noise. Each Pauli error occurs on some edge with probability p_i. Model this by log-ing of (14) to give each edge $w_i = \ln\left(\frac{1-p_i}{p_i}\right)$. Then error chains add weights along each edge.
4. The minimum weight case is the shortest path between boundary nodes. The shortest path contains at most 2 boundary nodes i.e. activated detectors.
5. At sufficiently low p_i under uniform depolarizing noise model, minimum weight edges dominate.

$\therefore$ We can constrain the decoding problem by only considering graphlike edges in space and time (graphlike = ≤ 2 nodes per edge). These graphlike edges represent the absolute minimum weight edge for a given syndrome.

Write the MWPM space+time incidence matrices:

$$H_G = \begin{pmatrix} H_X & 0 \\ 0 & H_Z \end{pmatrix} \text{. note the absence of the } \begin{matrix} H_X \\ H_Z \end{matrix} \text{ 4 column} \quad (18a)$$

$$H_{st} = \begin{pmatrix} H_G & & \\ & H_G & \\ & & \ddots \end{pmatrix} \begin{pmatrix} I & & \\ I & I & \\ & I & I \\ & & \ddots \end{pmatrix} \quad \text{where} \quad \begin{matrix} \rightarrow \text{time} \\ \downarrow \text{space} \end{matrix} \text{ sees edges from } t-1 \text{ only.} \quad (18b)$$

MWPM solves a slightly different version of the general decoding problem. 1) graphlike restriction 2) must select 1 single most likely c instead of summing over all c in each $l \in \{0,1\}$. (14) changes by applying $\mathcal{O}$ to the result of $\max_c$ instead of guiding the c in the sum and choosing $\max_l$ of sums of c.

$$\hat{l} = O \cdot \arg\max_c \prod_i p_i^{c_i} (1-p_i)^{1-c_i}, \quad H_G c = s \quad (19a)$$

$$= O \cdot \arg\min_c \sum_i w_i c_i, \quad w_i = \ln\left(\frac{1-p_i}{p_i}\right), \quad H_G c = s \quad (19b).$$

$O \cdot \arg\min_c$ gives a result $\hat{l} \in \{0,1\}$ directly instead of choosing $\hat{l}$ with highest value of c holistically. $H_G c = s$ condition is the graphlike constraint. w_i is the weight we minimize for determining most likely c's.

Since MWPM can only handle graphlike edges, it must decompose larger syndromes. Generally this is handled by assigning each decomposed edge the original total prob. and decoding individually. Let p_a be the probability of some hyperedge, let b and c be the graphlike decompositions of a.

$$a \mapsto b \wedge c$$

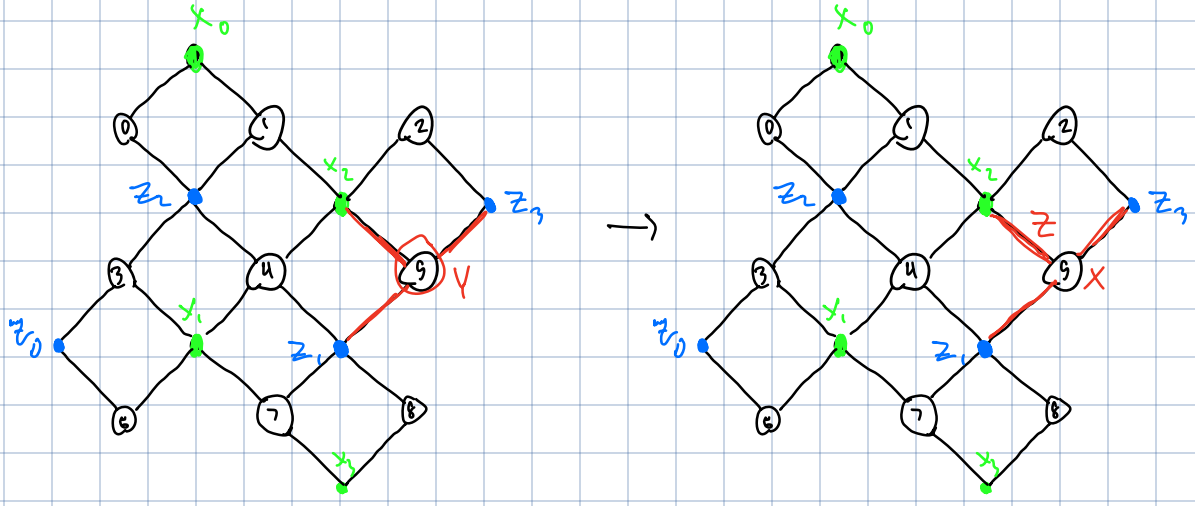
$p_a \mapsto p_a \wedge p_a = p_a^2$ under decomposition (20) → treated as second-order event

b and c already corresponded to some edge with p_b and p_c.

$$p_b' = p_b(1-p_a) + p_a(1-p_b) \simeq p_b + p_a$$
$$p_c' = p_c(1-p_a) + p_a(1-p_c) \simeq p_c + p_a \quad (21)$$

Every edge weight is skewed by this prior. So in the case that a does not fire, the edge $w_{b'} = \ln\left(\frac{1-p_b'}{p_b'}\right)$ is overweighted compared to $w_b = \ln\left(\frac{1-p_b}{p_b}\right)$ ∴ less probable.

Easy example: Y error → X Z.



In your case the Y edge is its own edge with a unique syndrome. Under MWPM the edge splits into 2 independent edges which have degenerate syndromes.

This edge decomposition represents the fundamental gap between MWPM and an ideal decoder. When measured against the expensive ideal computation, ~95% of the accuracy left on the table is due to hyperedge decomposition.

∴ This motivates a neural decoder to go beyond MWPM accuracy with comparable or lower latency.